

Universidad del Valle de Guatemala

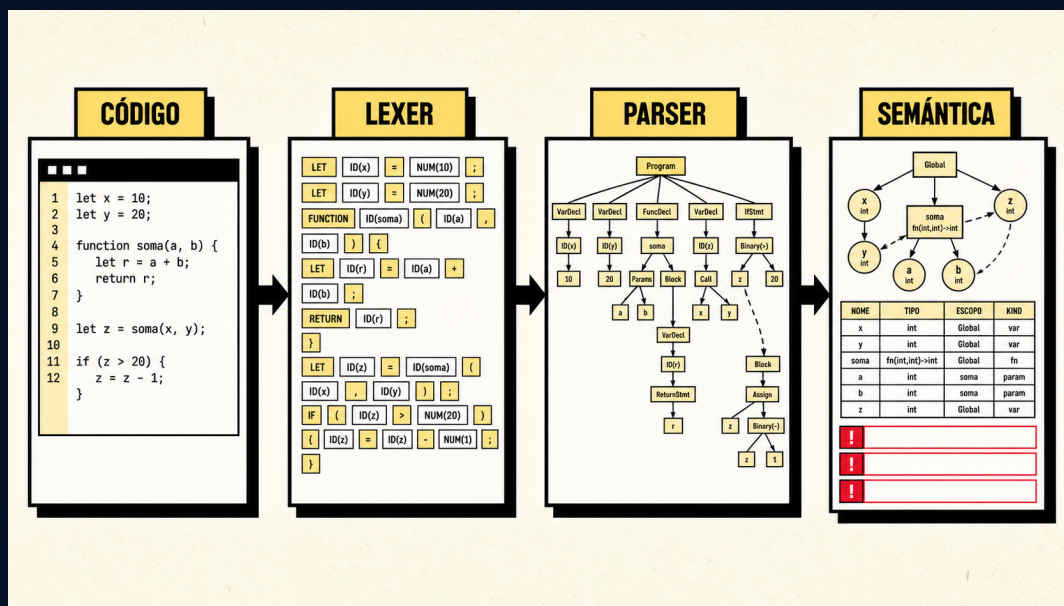
Facultad de Ingeniería

Construcción de Compiladores – CC-3032, sección 10

Compiscript Semantic IDE

Proyecto 1

Análisis semántico, tabla de símbolos e IDE



Informe técnico, teórico y práctico ampliado

Grupo DragonsSlayers 2.0

Pablo Daniel Barillas Moreno 22193

Hugo Daniel Barillas Ajín 23556

Ernesto Ascencio 23009

Catedrático: Ing. Carlos Valdéz

Agosto de 2026

Resumen

Este informe documenta integralmente el diseño y la construcción de **Compiscript Semantic IDE**, un entorno académico que integra las tres primeras fases del frente de un compilador: análisis léxico, análisis sintáctico y análisis semántico. La entrada es código fuente escrito en COMPIScript, un lenguaje educativo inspirado en TypeScript. La salida reúne tokens, diagnósticos localizados, árbol de parseo, árbol semántico anotado, tabla de símbolos, árbol de ámbitos, referencias y capturas de variables en closures.

El lexer y el parser son generados con ANTLR 4 mediante `antlr4ts`. La fase semántica recorre el árbol de parseo con un Visitor escrito en TypeScript. Esta fase aplica reglas de tipos, resolución de nombres, mutabilidad, funciones, clases, herencia, arreglos, control de flujo, inicialización y visibilidad. La arquitectura mantiene una separación estricta entre el motor y sus consumidores: la interfaz React, la herramienta de línea de comandos y la aplicación Electron utilizan la misma función de orquestación y el mismo contrato de resultado.

La documentación describe tanto la fundamentación teórica como las decisiones prácticas. Se detallan la gramática, el sistema de tipos, el modelo de símbolos, los algoritmos de resolución, el manejo de scopes, la inferencia de retornos, la identidad de clases, la detección de código inalcanzable, el catálogo **SEM001–SEM023** y el diseño de la experiencia de usuario. También se presentan diagramas vectoriales, ejemplos reproducibles, una matriz de trazabilidad y la estrategia de pruebas.

La versión documentada reporta **115 pruebas aprobadas en 7 suites**, generación reproducible de ANTLR, comprobación TypeScript y build de producción exitoso con **3368 módulos transformados**. El alcance concluye en el análisis semántico: no se interpreta el programa, no se genera código intermedio y no se produce código de máquina.

Palabras clave: compiladores, análisis semántico, tabla de símbolos, ANTLR 4, TypeScript, Visitor, ámbitos, sistema de tipos, IDE.

Abstract

This report presents the complete design and implementation of **Compiscript Semantic IDE**, an academic environment integrating lexical, syntactic and semantic analysis. ANTLR-generated recognizers transform source text into tokens and a concrete parse tree. A TypeScript Visitor then performs name resolution, type checking, scope management, control-flow validation, class and function analysis, and semantic-tree construction.

The system exposes a unified result model to a React web interface, a command-line client and an Electron desktop application. Its semantic infrastructure includes symbol insertion, lookup, controlled updates, nested scopes, references, closure captures, class identity, inheritance and return-type inference. The implementation is validated by 115 automated tests distributed across seven suites (dedicated lexer, parser and semantic testers, plus rubric regressions). This document combines theoretical background, implementation details, diagrams, reproducible examples, traceability and operational instructions.

Índice

Resumen	i
Abstract	ii
1 Introducción	1
1.1 Planteamiento del problema	1
1.2 Justificación	1
1.3 Objetivo general	1
1.4 Objetivos específicos	2
1.5 Alcance	2
1.6 Metodología de desarrollo	2
2 Fundamentos teóricos	3
2.1 El frente de un compilador	3
2.2 Tokens, lexemas y posiciones	3
2.3 Gramáticas libres de contexto	3
2.4 CST, AST y árbol semántico	4
2.5 Atributos y análisis semántico	4
2.6 Tabla de símbolos y resolución	4
2.7 Sistemas de tipos	4
2.8 Flujo de control	4
2.9 Patrón Visitor	5
3 Especificación de Compiscript	6
3.1 Fuentes de verdad	6
3.2 Unidades léxicas	6
3.3 Declaraciones	6

3.4	Expresiones y precedencia	7
3.5	Control, funciones, clases y arreglos	7
3.6	Decisiones interpretativas	8
4	Arquitectura general	9
4.1	Capas	9
4.2	Organización del repositorio	9
4.3	Pipeline de ejecución	10
4.4	Contrato de resultado	10
4.5	Tecnologías	11
4.6	Recursos visuales de referencia	11
5	Implementación léxica y sintáctica	13
5.1	Generación con ANTLR	13
5.2	Integración del lexer	13
5.3	Normalización de errores léxicos	14
5.4	Integración del parser	14
5.5	Filtrado de cascadas	14
5.6	Representaciones del árbol	15
5.7	Transición a semántica	15
6	Modelo semántico	16
6.1	Visión general	16
6.2	Prepasada y hoisting local	16
6.3	Árbol semántico anotado	16
6.4	Límite y deduplicación	17
7	Sistema de tipos	18
7.1	Representación	18
7.2	Igualdad y asignabilidad	18
7.3	Operadores aritméticos	19
7.4	Lógicos, relacionales y comparación	19
7.5	Asignación e inicialización	19
7.6	Arreglos	20

7.7	Funciones y retorno	20
7.8	Clases e identidad	20
8	Tabla de símbolos y ámbitos	21
8.1	Modelo de símbolo	21
8.2	Ámbitos	21
8.3	Insertar, recuperar y actualizar	22
8.4	Entrada y salida	22
8.5	Shadowing	22
8.6	Referencias	22
8.7	Closures	22
9	Reglas semánticas por construcción	24
9.1	Declaraciones de variables	24
9.2	Constantes	24
9.3	Resolución de identificadores	24
9.4	Asignaciones	25
9.5	Funciones	25
9.5.1	Registro de firmas	25
9.5.2	Scope y parámetros	25
9.5.3	Llamadas	25
9.5.4	Retornos	25
9.6	Clases	25
9.6.1	Identidad y declaración	25
9.6.2	Miembros	26
9.6.3	Herencia	26
9.6.4	Constructores e instanciación	26
9.6.5	Uso de this	26
9.7	Acceso a miembros	26
9.8	Arreglos	26
9.9	Condicionales	26
9.10	Ciclos	27
9.11	Switch	27
9.12	Try/catch	27

9.13 Break, continue y return	27
9.14 Código inalcanzable	27
10 Diagnósticos semánticos	29
10.1 Modelo de diagnóstico	29
10.2 Catálogo completo	29
10.3 Prevención de cascadas	30
10.4 Severidades	31
11 Diseño del IDE	32
11.1 Principios de experiencia de usuario	32
11.2 Jerarquía de componentes	33
11.3 Editor y carga de archivos	33
11.4 Navegación por fases	33
11.5 Explorador semántico	33
11.6 Tabla de símbolos	34
11.7 Árbol de ámbitos	34
11.8 Árbol semántico	34
11.9 Descargas	34
11.10Diseño adaptable	34
12 Interfaces de ejecución y distribución	35
12.1 Interfaz web	35
12.2 CLI	35
12.3 Electron	35
12.4 Empaquetado	35
12.5 Configuración reproducible	36
13 Pruebas y aseguramiento de calidad	37
13.1 Estrategia	37
13.2 Distribución	38
13.3 Pruebas de ScopeManager	38
13.4 Pruebas semánticas	38
13.5 Pruebas del pipeline	39

13.6 Ejemplos verificables	39
13.7 Comandos de verificación	39
13.8 Criterios de calidad	40
14 Trazabilidad con los requisitos	41
14.1 Reglas semánticas	41
14.2 Requisitos de construcción	42
14.3 Correspondencia ponderada	42
14.4 Contribuciones	42
15 Auditoría técnica y decisiones	43
15.1 Hallazgos corregidos	43
15.2 Decisión sobre float	43
15.3 Decisión sobre switch	44
15.4 Identidad de clases	44
15.5 Inferencia independiente del orden	44
15.6 Omisión de semántica tras errores previos	44
16 Instalación, uso y mantenimiento	45
16.1 Requisitos	45
16.2 Instalación	45
16.3 Uso del IDE	45
16.4 Uso de la CLI	46
16.5 Construcción	46
16.6 Modificación de la gramática	46
16.7 Modificación de una regla semántica	46
16.8 Compilación del informe	47
17 Resultados, limitaciones y trabajo futuro	48
17.1 Resultados obtenidos	48
17.2 Limitaciones	48
17.3 Rendimiento	49
17.4 Seguridad y privacidad	49
17.5 Trabajo futuro	49

18 Conclusiones	50
A Programa integral de demostración	51
B Ejemplos de diagnósticos	52
B.1 Tipos	52
B.2 Scopes	52
B.3 Funciones	52
B.4 Clases	52
B.5 Flujo y arreglos	53
C Inventario de módulos	54
D Inventario de comandos	56
E Glosario	57
F Ficha técnica	58

Índice de figuras

1.1	Ciclo incremental aplicado a cada familia de reglas.	2
2.1	Transformaciones principales del frente implementado.	3
3.1	Jerarquía de precedencia; la prioridad aumenta hacia abajo.	7
4.1	Arquitectura por capas.	9
4.2	Decisiones del orquestador.	10
4.3	Contrato serializable común.	10
4.4	Referencia visual neobrutalista del pipeline de Compiscript.	11
4.5	Referencia visual neobrutalista de ámbitos y tabla de símbolos.	12
5.1	Normalización de diagnósticos léxicos y sintácticos.	14
6.1	Colaboración de módulos semánticos.	16
7.1	Relaciones destacadas del sistema de tipos.	19
8.1	Ejemplo de árbol de ámbitos.	22
9.1	Flujo simplificado y detección de una instrucción inalcanzable.	28
10.1	Ciclo de vida de un diagnóstico semántico.	31
11.1	Jerarquía simplificada de componentes React del IDE neobrutalista.	33
13.1	Capas de la estrategia automatizada de pruebas.	37

Índice de cuadros

2.1	Representaciones internas complementarias.	4
3.1	Categorías léxicas principales.	6
4.1	Dependencias tecnológicas principales.	11
5.1	Información serializada de un token.	13
7.1	Familias del modelo de tipos.	18
7.2	Resultados aritméticos válidos.	19
8.1	Campos conceptuales de un símbolo.	21
10.1	Información contenida en un diagnóstico semántico.	29
10.2	Catálogo de diagnósticos semánticos.	29
12.1	Códigos de salida de la CLI.	35
13.1	Distribución de pruebas de la versión documentada.	38
13.2	Archivos de demostración semántica.	39
13.3	Atributos de calidad y evidencia concreta.	40
14.1	Trazabilidad de reglas semánticas.	41
14.2	Cobertura de requisitos de construcción.	42
14.3	Correspondencia resumida con la rúbrica del Proyecto 1.	42
15.1	Hallazgos y correcciones arquitectónicas.	43
C.1	Inventario técnico del núcleo.	54
F.1	Ficha resumida del producto.	58

Capítulo 1

Introducción

1.1 Planteamiento del problema

Reconocer que una secuencia de caracteres cumple una gramática es necesario, pero no suficiente para determinar si un programa tiene sentido. Un parser puede aceptar `total = "hola" + verdadero`; si la producción permite una asignación y una suma; sin embargo, el programa todavía puede violar reglas de tipos, referirse a identificadores inexistentes o usar una instrucción de control fuera de su contexto. Estas restricciones dependen de información que no está disponible en una regla sintáctica aislada.

El Proyecto 1 aborda esa brecha mediante una fase semántica observable. No se trata solo de emitir una lista de errores: el producto debe explicar qué símbolos existen, en qué ámbito viven, qué tipo posee cada expresión, cuáles declaraciones son referenciadas y cómo se relacionan las clases. A la vez, la herramienta debe ser usable desde una interfaz gráfica y conservar un núcleo suficientemente modular para pruebas automatizadas.

1.2 Justificación

Construir un analizador semántico obliga a conectar teoría formal con decisiones de ingeniería de software. La gramática define formas posibles, pero el sistema de tipos y la tabla de símbolos definen relaciones que atraviesan el árbol completo. Por ejemplo, comprobar una llamada requiere conocer la firma declarada, resolver el identificador de la función, analizar todos los argumentos y comparar sus tipos en orden.

El proyecto permite experimentar con un pipeline generado, hace visibles estructuras internas de un compilador, transforma requisitos en algoritmos comprobables y establece una base para interpretación o generación de código.

1.3 Objetivo general

Construir una fase de análisis semántico para COMPIScript que sea verificable mediante pruebas, observable desde un IDE y modular para servir como base de fases posteriores del compilador.

1.4 Objetivos específicos

1. Generar el lexer y parser desde una gramática ANTLR 4.
2. Integrar las fases léxica, sintáctica y semántica mediante un único orquestador.
3. Diseñar un sistema de tipos para primitivos, arreglos, funciones, clases e instancias.
4. Implementar inserción, recuperación, actualización y manejo de ámbitos.
5. Validar declaraciones, expresiones, llamadas, retornos, clases y control.
6. Registrar referencias y variables capturadas por funciones anidadas.
7. Construir un árbol semántico anotado y métricas de la fase.
8. Presentar tokens, árboles, símbolos y diagnósticos en una interfaz amigable.
9. Mantener ejemplos de aceptación y rechazo junto con pruebas de regresión.
10. Documentar arquitectura, decisiones, instalación y trazabilidad.

1.5 Alcance

El sistema recibe texto o un archivo `.cps`; genera tokens; valida la estructura; ejecuta reglas semánticas cuando las fases anteriores son válidas; y presenta resultados en web, CLI o escritorio. Se incluyen declaraciones, tipos, arreglos, funciones, closures, clases, herencia, acceso a miembros, condicionales, ciclos, `switch`, `try/catch`, `break`, `continue` y `return`.

Quedan fuera del alcance la ejecución de instrucciones, generación de representación intermedia, optimización, generación de máquina, enlazado externo y módulos.

1.6 Metodología de desarrollo

El trabajo siguió un ciclo incremental. Primero se alineó la gramática y se regeneró ANTLR. Después se diseñaron tipos, símbolos y ámbitos. Las reglas se incorporaron por familias y cada una recibió casos válidos, inválidos y regresiones. Finalmente se integró el resultado en la UI, se elaboraron ejemplos y se auditó la consistencia.

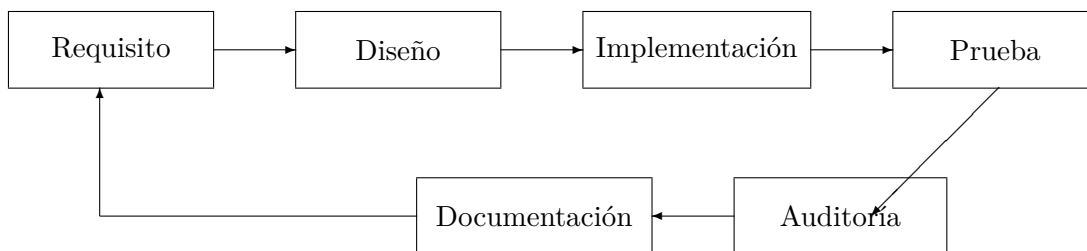


Figura 1.1: Ciclo incremental aplicado a cada familia de reglas.

Capítulo 2

Fundamentos teóricos

2.1 El frente de un compilador

El frente transforma texto en una representación estructurada y enriquecida. El lexer agrupa caracteres; el parser comprueba derivaciones; y el análisis semántico relaciona declaraciones y usos, calcula tipos y aplica restricciones contextuales.

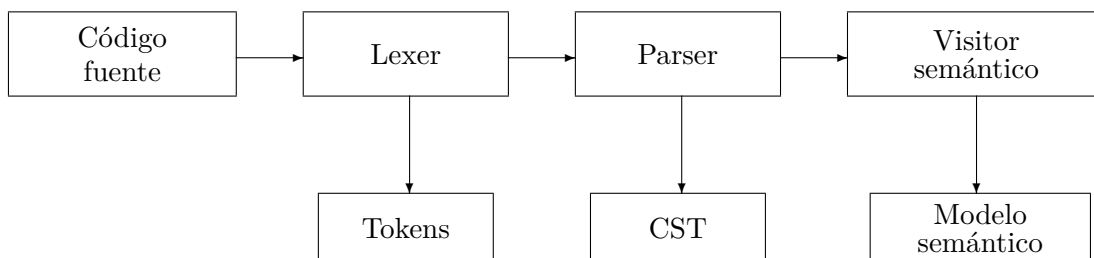


Figura 2.1: Transformaciones principales del frente implementado.

2.2 Tokens, lexemas y posiciones

Un lexema es el fragmento concreto de entrada; un token es su categoría. En `let total: integer = 5;; total` y `5` son lexemas, mientras `Identifier` e `IntegerLiteral` son tokens. Cada token conserva línea y columna; los símbolos guardan declaración y referencias; y los diagnósticos se vinculan a la construcción que los originó.

2.3 Gramáticas libres de contexto

Una gramática puede expresarse como $G = (V, \Sigma, P, S)$, donde V es el conjunto de no terminales, Σ el de terminales, P las producciones y S el símbolo inicial. `program` es la regla inicial. ANTLR 4 utiliza predicción adaptativa $LL(*)$ y genera un parser descendente; el proyecto no implementa tablas ACTION/GOTO.

2.4 CST, AST y árbol semántico

El CST conserva reglas auxiliares, puntuación y terminales. Un AST elimina detalles superficiales. Este proyecto mantiene el CST para trazabilidad y construye un árbol semántico anotado para explicar tipos, símbolos y diagnósticos.

Representación	Contenido	Uso
CST	reglas y terminales de ANTLR	validación sintáctica y depuración
Árbol semántico	nodos conceptuales, tipos y símbolos	explicación y UI
Tabla de símbolos	declaraciones, firmas y referencias	resolución contextual

Cuadro 2.1: Representaciones internas complementarias.

2.5 Atributos y análisis semántico

La semántica puede verse como el cálculo de atributos. Una expresión sintetiza su tipo a partir de sus hijos y una instrucción hereda el ámbito y el contexto:

$$\frac{\Gamma \vdash e_1 : T_1 \quad \Gamma \vdash e_2 : T_2 \quad \text{sumable}(T_1, T_2)}{\Gamma \vdash e_1 + e_2 : \text{resultado}(T_1, T_2)}.$$

Γ es el entorno de símbolos visible.

2.6 Tabla de símbolos y resolución

La resolución de un nombre n desde un ámbito s se define como:

$$\text{resolve}(n, s) = \begin{cases} \text{local}(n, s), & n \in s, \\ \text{resolve}(n, \text{parent}(s)), & \text{parent}(s) \neq \emptyset, \\ \perp, & \text{en otro caso.} \end{cases}$$

$\perp$ produce **SEM001**. La redeclaración consulta solo el ámbito actual; por ello se permite shadowing en ámbitos hijos.

2.7 Sistemas de tipos

Un tipo resume operaciones válidas. El proyecto combina clases nominales con arreglos y funciones estructurados. **unknown** representa información todavía no inferida y **error** absorbe consecuencias después de un diagnóstico. Se admite **integer** $\preceq$ **float**, pero no la conversión implícita inversa.

2.8 Flujo de control

El análisis distingue terminación del camino actual y retorno garantizado. Para un condicional completo:

$$\text{returns}(\text{if}(e) \ B_1 \ \text{else} \ B_2) = \text{returns}(B_1) \wedge \text{returns}(B_2).$$

Una instrucción posterior a `return`, `break` o `continue` en el mismo bloque es inalcanzable.

2.9 Patrón Visitor

El Visitor separa la estructura generada del comportamiento. La implementación no modifica el parser producido por ANTLR, por lo que la regeneración continúa siendo reproducible y las reglas semánticas permanecen en módulos TypeScript propios.

Capítulo 3

Especificación de Compiscript

3.1 Fuentes de verdad

La definición se construyó con el README, la gramática `src/grammars/Compiscript.g4` y el enunciado semántico. La gramática gobierna la sintaxis; el enunciado las validaciones; y los ejemplos aclaran intención cuando no contradicen las fuentes anteriores.

Idea clave. Los archivos de `src/generated` no se editan. Se modifica `Compiscript.g4` y se ejecuta `npm run generate`.

3.2 Unidades léxicas

Cuadro 3.1: Categorías léxicas principales.

Categoría	Elementos representativos
Declaraciones	<code>let</code> , <code>var</code> , <code>const</code> , <code>function</code> , <code>class</code>
Control	<code>if</code> , <code>else</code> , <code>while</code> , <code>for</code> , <code>foreach</code> , <code>switch</code>
Transferencia	<code>return</code> , <code>break</code> , <code>continue</code>
Objetos	<code>new</code> , <code>this</code>
Tipos	<code>boolean</code> , <code>integer</code> , <code>float</code> , <code>string</code>
Literales	enteros, decimales, cadenas, booleanos y <code>null</code>
Operadores	aritméticos, lógicos, relacionales, igualdad, asignación y ternario
Separadores	paréntesis, llaves, corchetes, coma, punto, dos puntos y punto y coma
Ignorados	espacios, saltos de línea y comentarios

3.3 Declaraciones

```
let contador: integer = 0;
```

```

var promedio: float = 87.5;
const NOMBRE: string = "Compiscript";
let activo: boolean = true;

function sumar(a: integer, b: integer): integer {
    return a + b;
}

```

3.4 Expresiones y precedencia

De menor a mayor prioridad aparecen asignación, ternario, OR, AND, igualdad, relacionales, suma/resta, multiplicación/división/módulo, unarios y primarias.

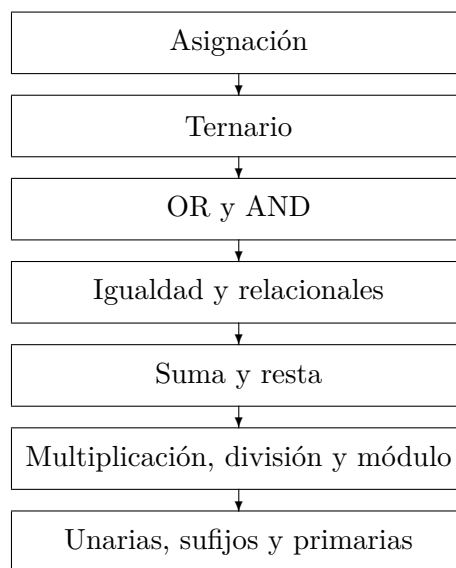


Figura 3.1: Jerarquía de precedencia; la prioridad aumenta hacia abajo.

3.5 Control, funciones, clases y arreglos

Se soportan condicionales, ciclos, `foreach`, `switch` y `try/catch`. La gramática activa exige bloques con llaves. Las funciones pueden ser recursivas o anidadas. Las clases admiten herencia, `this`, constructor y miembros. Los arreglos pueden ser multidimensionales.

```

class Caja {
    let valor: integer;
    function constructor(valor: integer) {
        this.valor = valor;
    }
    function obtener(): integer {
        return this.valor;
    }
}

```

```
}  
let cajas: Caja[] = [new Caja(10), new Caja(20)];  
print(cajas[0].obtener());
```

3.6 Decisiones interpretativas

`float` se añadió porque el requisito semántico lo exige. Se incorporaron tipo, literal decimal y promoción segura. Para `switch`, se adoptó un discriminante escalar y casos comparables, coherente con la gramática y los ejemplos oficiales.

Capítulo 4

Arquitectura general

4.1 Capas

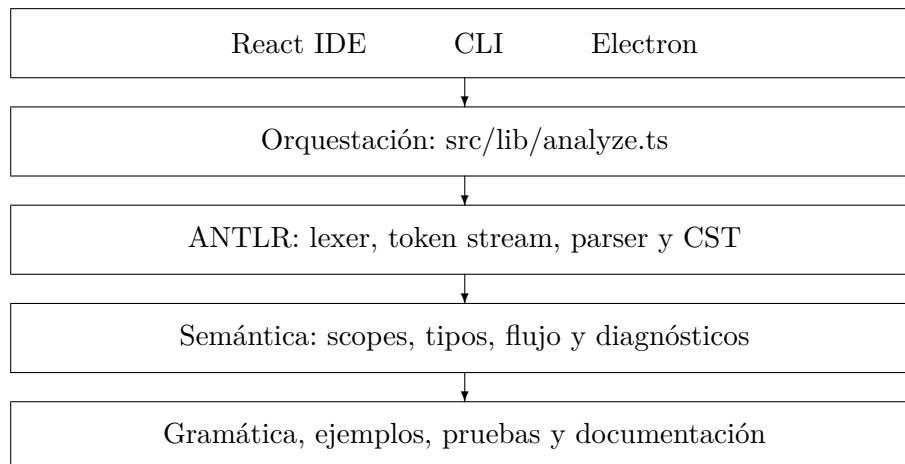


Figura 4.1: Arquitectura por capas.

4.2 Organización del repositorio

```
compiscript-ide-work/
|-- docs/                documentación e informe
|-- electron/main.cjs    proceso de escritorio
|-- examples/            casos léxicos, sintácticos y semánticos
|-- presentation/        presentación e imágenes
|-- public/              recursos del frontend
|-- src/
|   |-- __tests__/        siete suites Vitest
|   |-- cli/run.ts        cliente de consola
|   |-- generated/        salida de ANTLR
|   |-- grammars/         gramática fuente
```

```

| |-- lib/           orquestación
| |-- semantic/      fase semántica
| +-- ui/            aplicación React
|-- package.json
|-- tsconfig.json
+-- vite.config.ts

```

4.3 Pipeline de ejecución

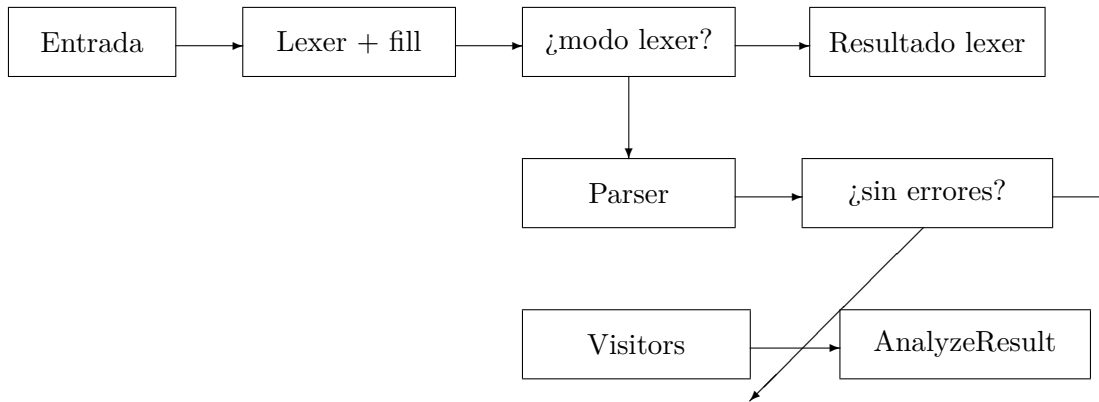


Figura 4.2: Decisiones del orquestador.

4.4 Contrato de resultado

`AnalyzeResult` contiene tokens, errores, tres representaciones del CST, resultado semántico, explicación y resumen:

$$\text{accepted} = \neg E_{\text{lex}} \wedge \neg E_{\text{syn}} \wedge (m \neq \text{semantic} \vee \neg E_{\text{sem}}).$$

Las advertencias no rechazan el archivo.

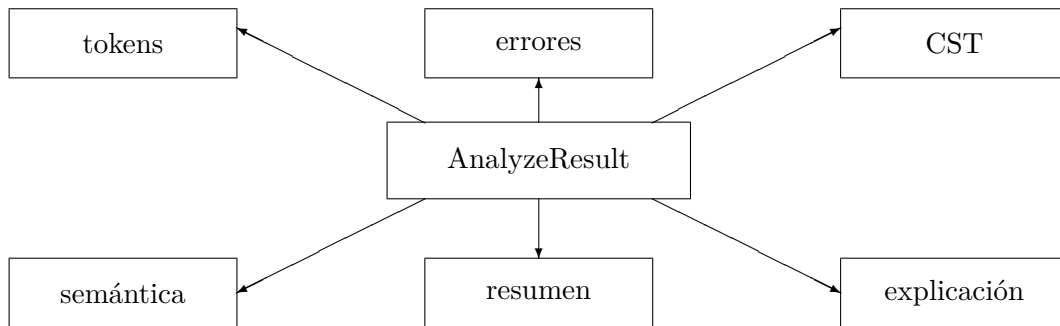


Figura 4.3: Contrato serializable común.

4.5 Tecnologías

Tecnología	Versión	Responsabilidad
TypeScript	5.6	motor y UI tipados
ANTLR / antlr4ts	4 / 0.5 alpha	lexer, parser, listener y Visitor
React	19	interfaz declarativa
Vite	6	desarrollo y build web
Vitest	3.2	pruebas
Electron	43	escritorio
electron-builder	26	empaquetado Windows, macOS y Linux
LaTeX + TikZ	TeX Live	documentación vectorial

Cuadro 4.1: Dependencias tecnológicas principales.

4.6 Recursos visuales de referencia

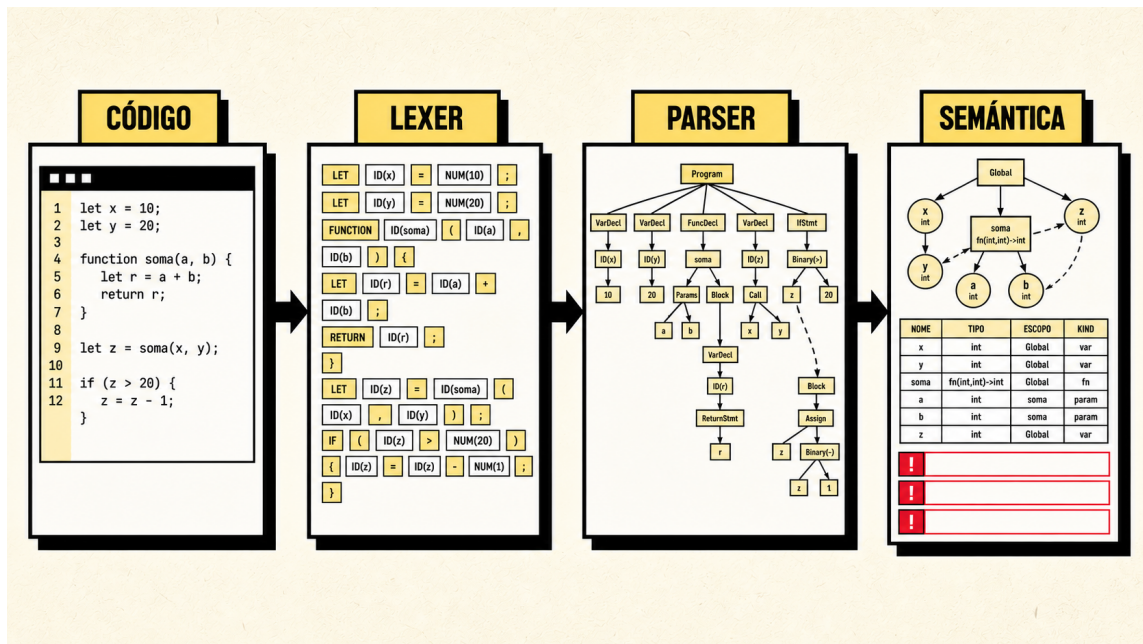


Figura 4.4: Referencia visual neobrutalista del pipeline de Compiscrypt.

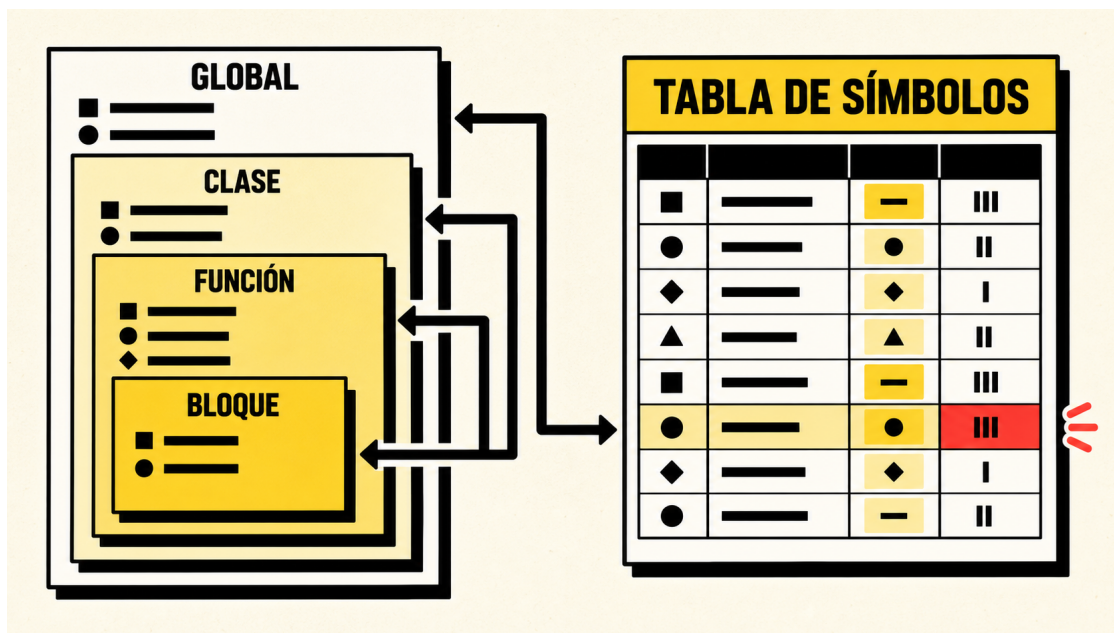


Figura 4.5: Referencia visual neobrutalista de ámbitos y tabla de símbolos.

Ambas piezas siguen la identidad visual neobrutalista de la interfaz vigente: fondos crema, superficies blancas y amarillas, bordes negros gruesos, esquinas rectas y sombras sólidas desplazadas. Funcionan como referencias conceptuales para explicar el sistema; no sustituyen las capturas ni los resultados producidos por una ejecución real del IDE.

Capítulo 5

Implementación léxica y sintáctica

5.1 Generación con ANTLR

El script `npm run generate` invoca `antlr4ts-cli` con generación de listener y Visitor. La salida se deposita en `src/generated`; incluye lexer, parser, interfaces y archivos auxiliares de vocabulario. La gramática permanece como fuente declarativa y el código generado puede reproducirse.

```
antlr4ts -Xexact-output-dir -visitor -listener
-o src/generated src/grammars/Compiscript.g4
```

Nunca se corrige un defecto editando `CompiscriptParser.ts`. Se modifica `Compiscript.g4`, se regenera y se ejecuta toda la suite.

5.2 Integración del lexer

`ANTLRInputStream` adapta el texto; `CompiscriptLexer` reconoce tokens; y `CommonTokenStream` almacena el resultado. `tokenStream.fill()` obliga al lexer a recorrer toda la entrada antes del parser. Así se recolectan múltiples errores léxicos y se conservan tokens posteriores.

Campo	Significado
<code>index</code>	posición dentro del token stream
<code>type</code>	identificador numérico generado
<code>typeName</code>	nombre simbólico del vocabulario
<code>text</code>	lexema exacto
<code>line</code>	línea base uno
<code>column</code>	columna convertida a base uno
<code>channel</code>	canal del token

Cuadro 5.1: Información serializada de un token.

5.3 Normalización de errores léxicos

El listener predeterminado fue sustituido por `CollectingErrorListener`. En lugar de escribir en consola, construye objetos localizables y traduce los mensajes. Fragmentos contiguos se agrupan: `@@@` produce un diagnóstico con ese símbolo, no tres mensajes equivalentes.

Las cadenas reciben tratamiento específico. Una comilla sin cierre produce una descripción de cadena incompleta; una secuencia de escape no permitida genera otro mensaje. La normalización conserva el fragmento ofensivo.

5.4 Integración del parser

Después de inspeccionar el token stream se ejecuta `seek(0)`. El parser agrega el listener recolector, habilita el árbol y llama `program()`.

```
tokenStream.seek(0);  
const parser = new CompiscriptParser(tokenStream);  
parser.errorHandler = new DefaultErrorStrategy();  
parser.buildParseTree = true;  
const tree = parser.program();
```

`DefaultErrorStrategy` puede insertar conceptualmente un token faltante, eliminar uno sobrante o sincronizarse con una regla posterior. El objetivo es evitar que una anomalía impida localizar problemas independientes.

5.5 Filtrado de cascadas

Cuando el lexer descarta un símbolo, el parser puede informar una consecuencia en la misma posición. El orquestador filtra únicamente errores sintácticos inmediatamente cercanos al fragmento léxico. Para cadenas inválidas se admite una única consecuencia en la línea siguiente.

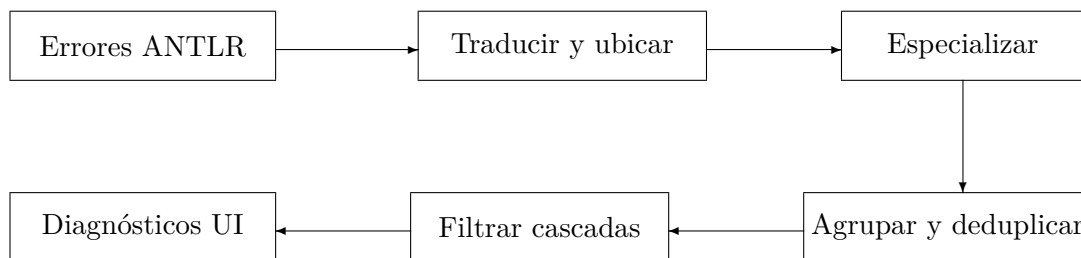


Figura 5.1: Normalización de diagnósticos léxicos y sintácticos.

5.6 Representaciones del árbol

El CST se conserva como salida Lisp original, recorrido indentado y nodos recursivos para la vista interactiva. La conversión recorre el objeto `ParseTree`; no reinterpreta la cadena Lisp, porque tokens como paréntesis la harían ambigua.

5.7 Transición a semántica

La fase semántica solo recibe un CST sin errores previos. Un nodo insertado por recuperación puede carecer de identificador o expresión real y generaría mensajes semánticos falsos. El resultado distingue `not-requested`, `skipped` y `completed`.

Capítulo 6

Modelo semántico

6.1 Visión general

La fase coopera con un administrador de ámbitos, un registro de clases, un sistema de tipos, un catálogo de diagnósticos y análisis de flujo.

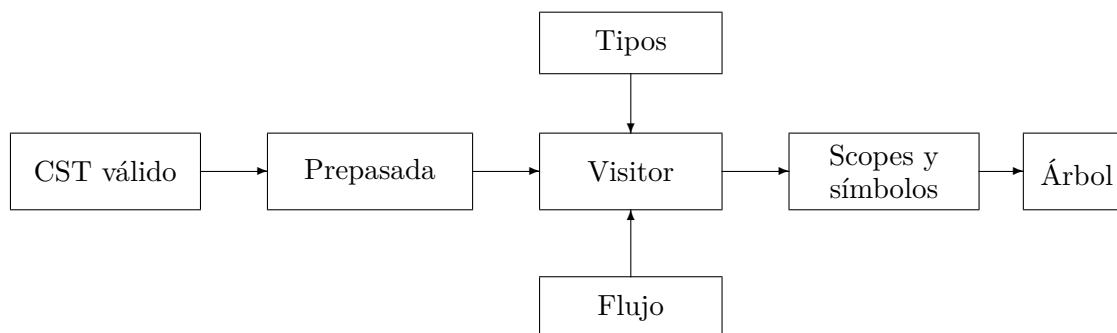


Figura 6.1: Colaboración de módulos semánticos.

6.2 Prepasada y hoisting local

Clases y funciones deben ser visibles antes de su aparición textual para permitir recursión y referencias adelantadas. Las declaraciones se registran antes de recorrer instrucciones ordinarias del mismo scope. El hoisting es local: una función anidada no se convierte en global.

La prepasada asigna identidad a clases, vincula herencia y prepara miembros. El Visitor entra después en cada cuerpo. Esta división separa descubrimiento y validación.

6.3 Árbol semántico anotado

`SemanticTreeNode` contiene ID, clase de nodo, etiqueta, ubicación, tipo opcional, símbolo opcional, diagnósticos e hijos. No es una IR ejecutable; es una proyección pedagógica que conecta CST y

significado.

6.4 Límite y deduplicación

El Visitor limita la salida a 500 diagnósticos semánticos. La creación centralizada asigna ID, severidad e información relacionada. Los tipos **unknown** y **error** reducen cascadas posteriores.

Capítulo 7

Sistema de tipos

7.1 Representación

`SemanticType` es una unión discriminada. Los primitivos incluyen `boolean`, `integer`, `float`, `string`, `null`, `void`, `unknown` y `error`. Arreglos, funciones, clases e instancias contienen estructura adicional.

Tipo	Información almacenada
Primitivo	nombre canónico
Arreglo	tipo de elemento y dimensiones
Función	parámetros y retorno
Clase	nombre e identidad de declaración
Instancia	clase asociada e identidad
<code>unknown</code>	inferencia pendiente
<code>error</code>	causa ya diagnosticada

Cuadro 7.1: Familias del modelo de tipos.

7.2 Igualdad y asignabilidad

`typesEqual` compara estructura e identidad. Dos clases homónimas en ámbitos distintos no son iguales. `isAssignable` incorpora igualdad, promoción numérica, `null` donde aplica e herencia.

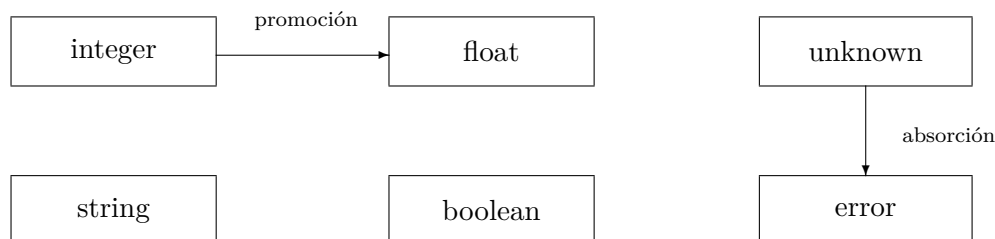


Figura 7.1: Relaciones destacadas del sistema de tipos.

7.3 Operadores aritméticos

`numericResult` devuelve `float` si algún operando es decimal; en otro caso devuelve `integer`. La suma admite concatenación cuando ambos operandos son cadenas. Aplicar aritmética a categorías inválidas produce **SEM004**.

Izquierdo	Derecho	Resultado
integer	integer	integer
integer	float	float
float	integer	float
float	float	float
string	string, solo suma	string

Cuadro 7.2: Resultados aritméticos válidos.

7.4 Lógicos, relacionales y comparación

Los operadores lógicos requieren booleanos. Las comparaciones de orden requieren números compatibles. Igualdad y desigualdad utilizan `isComparable`, que considera igualdad, promoción, `null` y jerarquía. Toda comparación válida resulta `boolean`.

7.5 Asignación e inicialización

Una variable anotada adopta ese tipo y el inicializador debe ser assignable. Sin anotación, se infiere desde el valor. Una variable sin inicializador queda pendiente; usarla produce **SEM023**. Las constantes son inmutables.

```

let entero: integer = 4;
let decimal: float = entero;
let pendiente: integer;
print(pendiente);           // SEM023
  
```

```
pendiente = 10;  
print(pendiente);           // valido
```

7.6 Arreglos

Un literal analiza todos sus elementos y calcula `commonType`. Si no existe tipo compatible se produce **SEM017**. Acceder exige receptor arreglo e índice entero: un índice decimal produce **SEM015**; indexar un escalar, **SEM016**.

7.7 Funciones y retorno

Una firma conserva parámetros y retorno. Sin anotación, cada `return` aporta un tipo observado y al finalizar se calcula un tipo común. Sin retornos con valor, el resultado es `void`. Una anotación incompatible produce **SEM008**.

7.8 Clases e identidad

La igualdad nominal usa `classId`, no solo nombre. Dos clases `Local` en bloques hermanos son tipos distintos. La identidad permite recorrer herencia y asignar una instancia hija a una variable del padre.

Capítulo 8

Tabla de símbolos y ámbitos

8.1 Modelo de símbolo

Cada `SymbolEntry` contiene ID, nombre, categoría, tipo, mutabilidad, scope, declaración, inicialización, referencias y captura. Funciones agregan firma; clases y miembros conservan identidad.

Cuadro 8.1: Campos conceptuales de un símbolo.

Campo	Propósito
ID	identidad estable
Nombre	texto sujeto a resolución
Categoría	variable, constante, parámetro, función, clase, campo o método
Tipo	declarado o inferido
Mutabilidad	controla reasignación
Scope ID	entorno propietario
Declaración	línea y columna originales
Inicialización	uso antes de valor
Referencias	ubicaciones de uso
Captura	uso desde una función anidada
Firma/miembros	metadatos compuestos

8.2 Ámbitos

`ScopeKind` distingue `global`, `block`, `function`, `class`, `loop`, `catch` y `switch`. Cada scope conoce padre, hijos, nombre, límites y símbolos.

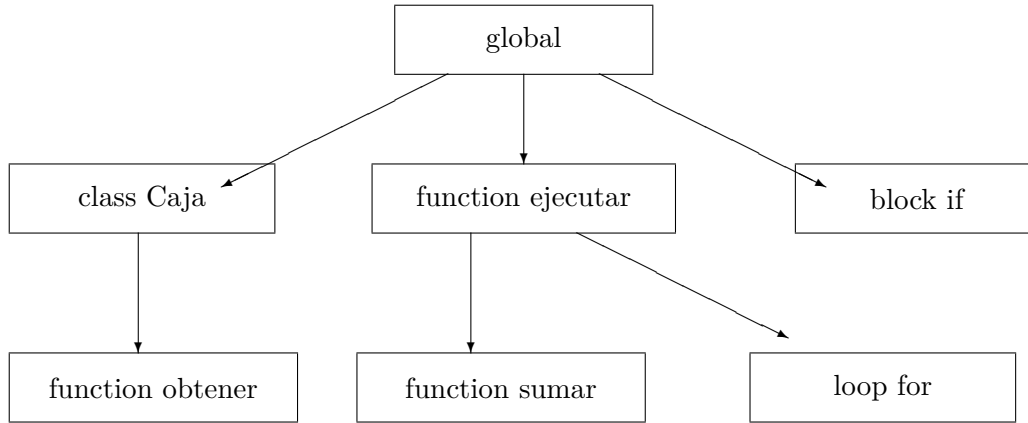


Figura 8.1: Ejemplo de árbol de ámbitos.

8.3 Insertar, recuperar y actualizar

`declare` consulta el scope activo; una colisión permite emitir **SEM002**. `resolveCurrent` limita la búsqueda; `resolve` recorre padres. `updateSymbol` permite cambiar tipo, firma, inicialización, captura y referencias, pero preserva identidad, nombre, scope y declaración.

8.4 Entrada y salida

`enterScope` crea un hijo y lo activa. `exitScope` fija su final y restaura el padre. La UI conserva scopes terminados:

$$S_0 = [global], \quad enter(k) \Rightarrow S_{i+1} = S_i \cdot k, \quad exit \Rightarrow S_{i+1} = parent(top(S_i)).$$

8.5 Shadowing

Un hijo puede repetir el nombre de un padre. Dentro del hijo se resuelve la entidad más cercana; al salir vuelve la del padre. Ambas conservan IDs y referencias independientes.

8.6 Referencias

Cada resolución registra la ubicación del uso, incluso clases en `new`, campos y métodos. Esto habilita métricas y una futura navegación a declaración.

8.7 Closures

Si una función anidada resuelve un símbolo de una función externa, se marca `captured`. Una variable global no se clasifica como captura.

```
function crearContador(): integer {  
  let base: integer = 40;  
  function incrementar(): integer {  
    return base + 1;  
  }  
  return incrementar();  
}
```

Capítulo 9

Reglas semánticas por construcción

9.1 Declaraciones de variables

El Visitor analiza primero la anotación y el inicializador. Si ambos existen, verifica asignabilidad; si solo hay inicializador, infiere; si solo hay anotación, registra la variable como no inicializada. Finalmente llama `declare`. Este orden permite que la entrada final conserve el tipo más útil incluso cuando se reporta un error.

Una redeclaración en el mismo scope produce **SEM002** y agrega información sobre la ubicación original. Declarar el mismo nombre en un hijo es shadowing válido.

9.2 Constantes

La gramática exige inicializador. La semántica valida el tipo de ese valor y registra mutabilidad falsa. Una asignación posterior a una constante se rechaza con **SEM003**. Las constantes de clase siguen la misma regla y forman parte del registro de miembros.

9.3 Resolución de identificadores

Al encontrar un identificador, el Visitor:

1. consulta la cadena léxica;
2. si no existe, emite **SEM001** y devuelve `error`;
3. si es variable no inicializada, emite **SEM023**;
4. registra la referencia;
5. comprueba si la referencia constituye una captura;
6. devuelve tipo y `symbolId`.

El tipo absorbente evita que `desconocido + 1` emita además un segundo mensaje por el operador.

9.4 Asignaciones

El lado izquierdo debe ser una referencia asignable: identificador mutable, campo mutable o posición de arreglo. Se analiza el lado derecho y se aplica `isAssignable`. Una asignación exitosa marca inicialización cuando el receptor es una variable pendiente.

9.5 Funciones

9.5.1 Registro de firmas

Antes del cuerpo se resuelven parámetros y retorno anotado. La declaración se inserta en el scope propietario para habilitar recursión. Dos funciones homónimas en el mismo scope producen [SEM002](#), porque el lenguaje no implementa sobrecarga.

9.5.2 Scope y parámetros

El cuerpo se analiza dentro de un scope `function`. Cada parámetro se declara como símbolo inicializado. Un nombre de parámetro repetido produce [SEM019](#); la ubicación permite señalar la colisión exacta.

9.5.3 Llamadas

Una llamada exige receptor de tipo función. Se compara aridad y luego cada argumento:

$$validCall(f, a_1, \dots, a_n) \iff n = |params(f)| \wedge \bigwedge_{i=1}^n assignable(type(a_i), type(p_i)).$$

La aridad incorrecta produce [SEM006](#); un tipo incompatible, [SEM007](#); invocar un valor no invocable, [SEM014](#). Aun con error se analizan todos los argumentos para encontrar problemas independientes.

9.5.4 Retornos

`return` fuera de función produce [SEM009](#). Dentro de una función anotada, el valor debe ser asignable al retorno y una discrepancia genera [SEM008](#). En una función no anotada, los tipos observados alimentan la inferencia.

9.6 Clases

9.6.1 Identidad y declaración

Cada clase recibe `classId`. El nombre visible se registra en `ScopeManager`; no existe un mapa global que haga visibles clases locales fuera de su bloque. Esta decisión corrige fugas y colisiones entre homónimos.

9.6.2 Miembros

Campos, constantes y métodos se recopilan antes del recorrido completo. Se procesan inicializadores de campos antes que métodos para que un método pueda usar un campo declarado después en el texto sin que la inferencia dependa del orden. El árbol semántico restaura luego el orden fuente para legibilidad.

9.6.3 Herencia

La clase padre debe existir y ser visible. Se detectan ciclos recorriendo la cadena de padres; una herencia inválida o circular produce [SEM020](#). La búsqueda de campo o método consulta primero la clase actual y después sus ancestros.

9.6.4 Constructores e instanciación

El método llamado `constructor` define la firma de inicialización. Si no existe, se utiliza un constructor implícito sin parámetros. `new` exige que el identificador resuelva una clase y que argumentos coincidan con la firma; los fallos usan [SEM006](#), [SEM007](#) o [SEM014](#).

9.6.5 Uso de `this`

`this` solo es válido dentro del contexto de una clase. Fuera de él produce [SEM013](#). Dentro de un método sintetiza el tipo de instancia correspondiente a la identidad de la clase actual.

9.7 Acceso a miembros

Para `objeto.miembro`, el receptor debe ser instancia. La búsqueda respeta herencia. Un miembro inexistente produce [SEM012](#); un campo aporta su tipo; un método aporta tipo función. La referencia se registra sobre el símbolo real del miembro, no únicamente sobre el objeto receptor.

9.8 Arreglos

`arrayLiteral` calcula el tipo común de todos los elementos. Los sufijos de índice reducen una dimensión. `foreach` exige receptor arreglo, crea un scope de ciclo y declara el iterador con el tipo de elemento.

9.9 Condicionales

`if`, `while`, `do-while` y la condición de `for` requieren `boolean`; una categoría distinta produce [SEM005](#). Cada cuerpo abre scope. Las ramas de un `if` se analizan de manera independiente.

9.10 Ciclos

El contexto incrementa la profundidad de ciclo para validar `break` y `continue`. El inicializador de `for` vive en un scope propio que abarca condición, actualización y cuerpo. `continue` fuera de ciclo produce [SEM011](#).

9.11 Switch

El discriminante debe ser escalar. Cada caso se compara con él mediante `isComparable`; un discriminante inválido o caso incompatible produce [SEM021](#). `break` es válido dentro del switch porque existe un contexto interrumpible aunque no sea un ciclo.

9.12 Try/catch

El bloque `try` se analiza normalmente. `catch` crea un scope propio y declara el identificador de error, evitando que escape fuera del manejador. La fase no modela excepciones dinámicas; únicamente alcance y estructura.

9.13 Break, continue y return

`break` requiere ciclo o switch; de lo contrario produce [SEM010](#). `continue` requiere ciclo. `return` requiere función. Estas reglas son contextuales y justifican mantener pilas explícitas durante el recorrido.

9.14 Código inalcanzable

`findFirstUnreachableIndex` identifica la primera instrucción posterior a un terminador del bloque. Cada instrucción inalcanzable recibe la advertencia [SEM018](#). Se usa severidad warning porque el programa conserva significado tipado, aunque contiene una ruta imposible.

```
function ejemplo(x: boolean): integer {
  if (x) {
    return 1;
    print("nunca");      // SEM018
  } else {
    return 2;
  }
}
```

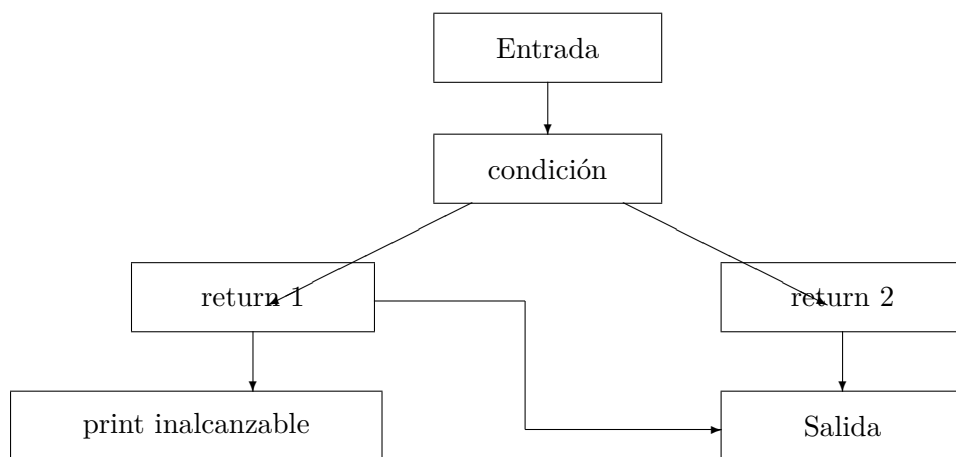


Figura 9.1: Flujo simplificado y detección de una instrucción inalcanzable.

Capítulo 10

Diagnósticos semánticos

10.1 Modelo de diagnóstico

Cada `SemanticDiagnostic` contiene ID, código, severidad, línea, columna, mensaje, sugerencia opcional, símbolo e información relacionada. El código hace que las pruebas no dependan de comparar textos completos; el mensaje permanece orientado al usuario.

Propiedad	Uso
<code>code</code>	identifica establemente la regla
<code>severity</code>	error o advertencia
<code>line, column</code>	localización base uno
<code>message</code>	explicación en español
<code>suggestion</code>	posible acción correctiva
<code>symbol</code>	entidad o lexema relacionado
<code>related</code>	ubicación adicional, como declaración previa

Cuadro 10.1: Información contenida en un diagnóstico semántico.

10.2 Catálogo completo

Cuadro 10.2: Catálogo de diagnósticos semánticos.

Código	Regla	Situación representativa
SEM001	Identificador no declarado	uso de nombre no visible
SEM002	Redeclaración	nombre repetido en el mismo scope
SEM003	Asignación incompatible	inicialización, reasignación o constante
SEM004	Operador inválido	operandos sin sentido para el operador
SEM005	Condición no booleana	if o ciclo con tipo diferente

Código	Regla	Situación representativa
SEM006	Aridad incorrecta	llamada o constructor con cantidad inválida
SEM007	Argumento incompatible	argumento no asignable al parámetro
SEM008	Retorno incompatible	valor no asignable al retorno
SEM009	Return contextual	return fuera de función
SEM010	Break contextual	break fuera de ciclo o switch
SEM011	Continue contextual	continue fuera de ciclo
SEM012	Miembro inexistente	propiedad o método ausente
SEM013	This contextual	this fuera de clase
SEM014	Invocación inválida	receptor no invocable o clase incorrecta
SEM015	Índice no entero	acceso con índice decimal, cadena u otro tipo
SEM016	Receptor no indexable	uso de corchetes sobre escalar
SEM017	Arreglo heterogéneo	elementos sin tipo común
SEM018	Código inalcanzable	instrucción posterior a terminador
SEM019	Parámetro duplicado	nombre repetido en una firma
SEM020	Herencia inválida	padre inválido o ciclo
SEM021	Switch incompatible	discriminante o case no comparable
SEM022	Reservado	compatibilidad del catálogo; sin regla activa
SEM023	Uso sin inicialización	variable visible pero todavía sin valor

10.3 Prevención de cascadas

La estrategia combina:

- omisión completa de semántica si lexer o parser fallan;
- tipos absorbentes para expresiones ya diagnosticadas;
- deduplicación por código y ubicación;
- registro del símbolo original en una redeclaración;
- límite máximo de diagnósticos;
- continuación del recorrido para hallar causas independientes.

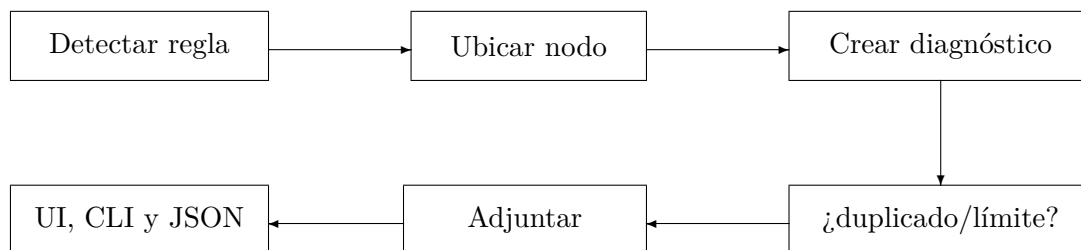


Figura 10.1: Ciclo de vida de un diagnóstico semántico.

10.4 Severidades

Los errores participan en la decisión de aceptación. Las advertencias señalan construcciones sospechosas que no impiden tipar el programa; actualmente **SEM018** se utiliza para código inalcanzable. La UI muestra conteos separados.

Capítulo 11

Diseño del IDE

11.1 Principios de experiencia de usuario

La interfaz está pensada como banco de trabajo de compiladores. El editor permanece como superficie dominante; la gramática puede plegarse; y los resultados se agrupan por fase. El usuario no necesita interpretar una excepción de ANTLR ni buscar un símbolo dentro de JSON crudo.

Los principios aplicados son:

1. revelar complejidad de forma progresiva;
2. mantener visible el estado del pipeline;
3. separar errores por fase;
4. permitir navegar representaciones grandes;
5. conservar acciones de copia y descarga;
6. usar terminología coherente entre UI, CLI y documentos.

11.2 Jerarquía de componentes

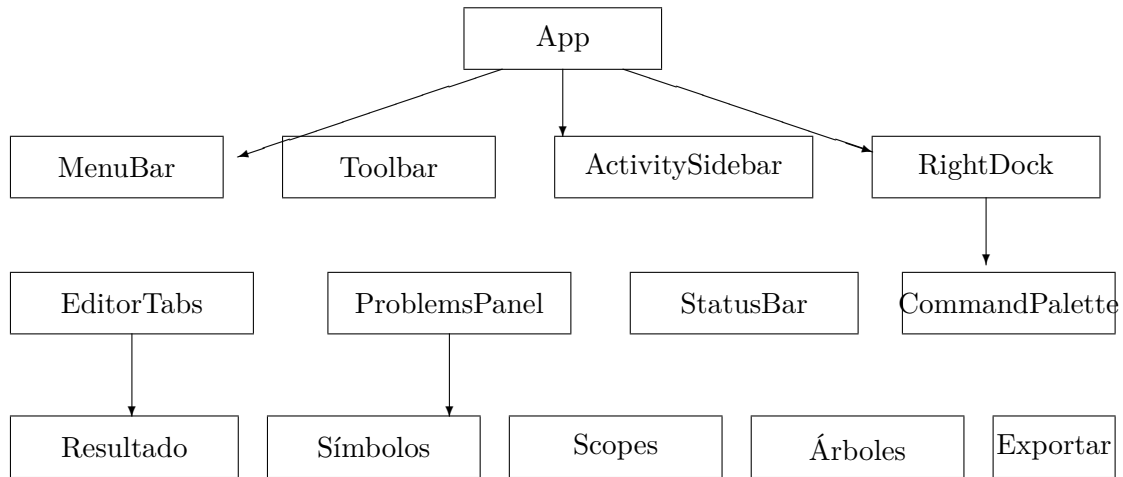


Figura 11.1: Jerarquía simplificada de componentes React del IDE neobrutalista.

11.3 Editor y carga de archivos

EditorTabs contiene el editor Monaco con resaltado de sintaxis COMPIScript, carga de `.cps`, edición, limpieza, copia y descarga. Se muestran nombre, cantidad de líneas, caracteres, posición del cursor y estado de preparación.

11.4 Navegación por fases

La barra lateral **ActivitySidebar** separa ejemplos y ruta de análisis. El dock derecho **RightDock** organiza resultado, símbolos, ámbitos, árboles, documentación y exportación. Esto permite estudiar tokenización, observar el CST, inspeccionar el pipeline completo o consultar la documentación sin cambiar de vista.

11.5 Explorador semántico

RightDock organiza resultados en:

Resumen. Métricas, aceptación y salud de la fase.

Diagnósticos. Filtros, severidad, código y ubicación.

Símbolos. Nombre, categoría, tipo, scope, mutabilidad y referencias.

Ámbitos. Árbol jerárquico de scopes.

Árboles. Comparación del CST con el árbol semántico.

11.6 Tabla de símbolos

`SymbolTablePanel` ofrece búsqueda y categorías. Los IDs permiten vincular declaraciones y usos; la marca de captura destaca closures; y los tipos se renderizan mediante una representación legible, no el objeto interno.

11.7 Árbol de ámbitos

`ScopeTreePanel` parte de `scopeRootId` y recorre hijos. Cada nodo muestra clase de scope, nombre, cantidad de símbolos y rango fuente. El usuario puede entender por qué un nombre es visible o por qué dos homónimos son distintos.

11.8 Árbol semántico

`SemanticTreePanel` representa nodos con tipo, referencia y diagnósticos. La vista es complementaria al CST: una muestra cómo la gramática reconoció el texto; la otra, qué significado calculó el Visitor.

11.9 Descargas

El frontend genera blobs locales para entrada, gramática, tokens CSV/JSON, árbol y resultado integral. No envía el código a un servidor. Los formatos estructurados facilitan auditoría externa y análisis posterior.

11.10 Diseño adaptable

Las cuadrículas cambian en anchos reducidos; tablas y árboles habilitan desplazamiento; bloques de código ajustan líneas; y botones conservan área táctil. La iconografía utiliza componentes vectoriales de Lucide en lugar de emojis.

Capítulo 12

Interfaces de ejecución y distribución

12.1 Interfaz web

Vite proporciona servidor de desarrollo y build. La aplicación se inicia con `npm run dev` y, por configuración, escucha en el puerto 3000. El build combina `tsc -noEmit` y `vite build`, de modo que un error de tipos impide producir una distribución aparentemente correcta.

12.2 CLI

`src/cli/run.ts` acepta ruta y modo. Reutiliza `analyzeInput`, imprime resumen y diagnósticos, y devuelve un código de salida:

Código	Significado
0	entrada aceptada en el modo solicitado
1	entrada rechazada o argumentos inválidos
2	fallo inesperado de infraestructura

Cuadro 12.1: Códigos de salida de la CLI.

12.3 Electron

`electron/main.cjs` crea la ventana y carga `dist`. La aplicación mantiene integración de Node desactivada, aislamiento de contexto y sandbox. Esto reduce la superficie expuesta por el contenido renderizado.

12.4 Empaquetado

`npm run exe:portable` genera una aplicación independiente y `npm run exe:installer` un instalador NSIS, ambos Windows x64. `npm run exe:mac` produce `.zip` y `.dmg` sin firma para Intel y Apple

Silicon, y `npm run exe:linux` produce un **AppImage** x64. Los tres se generan sin certificado comercial de firma, igual que la distribución portable de Windows. Los artefactos quedan en **release**. El usuario final no necesita Node ni Java; Java solo interviene al regenerar ANTLR.

Los targets macOS se construyen en una Mac o en un runner macOS. El AppImage se construye en Linux nativo, WSL o Docker; intentar producirlo directamente sobre NTFS puede fallar cuando la herramienta crea enlaces simbólicos. Esta separación conserva los permisos y utilidades propias de cada plataforma.

El último corte portable publicado para Windows es Compiscript Semantic IDE V1.2.0: https://github.com/DanielBarillasM/Proyecto-01_CDC_seccion-10_Grupo-DragonsSlayers-2.0/releases/tag/Compiscript-Semantic-IDE-V1.2.0. La rama **main** contiene cambios posteriores al tag, incluidos el panel de pruebas, los testers separados por fase y el empaquetado macOS/Linux; para obtener exactamente ese estado debe construirse desde la fuente vigente.

El arranque fue verificado en macOS arm64 mediante Electron directo y en Linux arm64 dentro de un contenedor con Xvfb. Los avisos de D-Bus en ambientes mínimos son benignos. Si un AppImage informa que falta **libz.so**, debe instalarse o exponerse la variante de desarrollo de zlib; los escritorios Linux habituales ya la incluyen.

12.5 Configuración reproducible

`package-lock.json` fija dependencias de npm. La gramática y el código generado están versionados. `tsconfig.json` centraliza TypeScript y `vite.config.ts` integra React y polyfills requeridos por antlr4ts en navegador.

Capítulo 13

Pruebas y aseguramiento de calidad

13.1 Estrategia

La validación se distribuye en capas. Las pruebas unitarias ejercitan **ScopeManager** y reglas de tipos. Las pruebas semánticas construyen programas pequeños con una intención precisa. Las pruebas del pipeline verifican la integración ANTLR–Visitor. Finalmente, los ejemplos de proyecto se leen desde disco para garantizar que la demostración siga sincronizada.

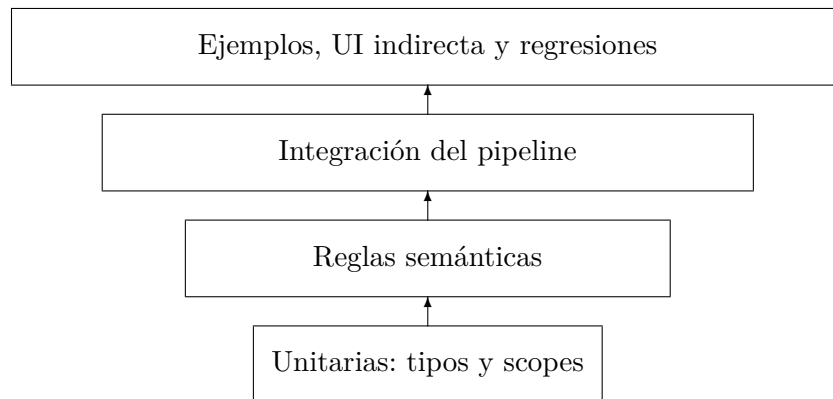


Figura 13.1: Capas de la estrategia automatizada de pruebas.

13.2 Distribución

Suite	Cantidad	Cobertura principal
<code>semanticAnalyzer.test.ts</code>	68	reglas semánticas y regresiones
<code>scopeManager.test.ts</code>	4	insertar, recuperar, actualizar y scopes
<code>projectExamples.test.ts</code>	8	archivos semánticos de demostración
<code>lexer.test.ts</code>	5	tokenización y diagnósticos léxicos
<code>parser.test.ts</code>	9	aceptación, CST y diagnósticos sintácticos
<code>rubric.examples.test.ts</code>	8	matriz léxica/sintáctica heredada
<code>testCases.defaults.test.ts</code>	13	valida los casos por defecto del panel "Pruebas" del IDE
Total	115	siete suites aprobadas

Cuadro 13.1: Distribución de pruebas de la versión documentada.

La suite histórica `examples.sync.test.ts` se retiró cuando los ejemplos integrados dejaron de duplicarse en cadenas TypeScript y pasaron a importarse directamente desde los archivos `.cps` mediante `?raw`. El antiguo `analyze.compiscript.test.ts` se dividió en `lexer.test.ts` y `parser.test.ts` para que cada fase tenga un tester identificable por separado, y `testCases.defaults.test.ts` se agregó para validar los casos por defecto del nuevo panel "Pruebas" del IDE, que expone estos mismos testers de forma interactiva sin salir de la aplicación.

13.3 Pruebas de ScopeManager

Las cuatro operaciones solicitadas se comprueban directamente:

1. insertar y recuperar en el scope activo;
2. resolver símbolos de padres y permitir shadowing;
3. actualizar información sin cambiar identidad;
4. entrar y salir de scopes restaurando el entorno correcto.

Estas pruebas aíslan la tabla del Visitor. Si falla resolución, puede determinarse si la causa está en la estructura de scopes o en la construcción que la usa.

13.4 Pruebas semánticas

Los 68 casos cubren aceptación y rechazo para operadores, asignación, inicialización, arreglos, llamadas, retornos, recursión, closures, control, miembros, constructores, herencia, identidad de clases, inferencia y referencias. El objetivo no es únicamente producir todos los códigos; cada regla tiene contexto válido para prevenir falsos positivos.

13.5 Pruebas del pipeline

Estas pruebas invocan `analyzeInput`. Verifican que lexer y parser sean los generados, que el modo lexer no ejecute parser, que semántica se omita ante errores previos, que los conteos del resumen correspondan a las listas y que árboles/tokens se produzcan.

13.6 Ejemplos verificables

Cuadro 13.2: Archivos de demostración semántica.

Archivo	Propósito
<code>valid_complete.cps</code>	programa integral válido
<code>symbol_table_demo.cps</code>	scopes, actualización, referencias y closures
<code>semantic_errors.cps</code>	demostración rápida de errores variados
<code>errors_types.cps</code>	asignación, operadores, condiciones y arreglos
<code>errors_scopes.cps</code>	nombres, redeclaración e inicialización
<code>errors_functions.cps</code>	aridad, argumentos, retorno e invocación
<code>errors_flow.cps</code>	contexto, código muerto y switch
<code>errors_classes.cps</code>	miembros, this, constructores y herencia
<code>errors_arrays.cps</code>	homogeneidad, índices y receptor

Los archivos `errors\_*.cps` tienen sintaxis válida: su rechazo debe provenir de la fase semántica. `projectExamples.test.ts` comprueba esa precondition y los códigos mínimos esperados.

13.7 Comandos de verificación

```
npm install
npm run generate
npm run check
npm test
npm run build
```

En la revisión documentada, generación, chequeo y pruebas terminan correctamente. El build transforma más de dos mil módulos. Vite emite una advertencia no bloqueante porque el bundle principal supera 500 kB.

13.8 Criterios de calidad

Criterio	Evidencia
Corrección	reglas y 115 pruebas
Reproducibilidad	gramática, lockfile y scripts
Trazabilidad	códigos, ubicaciones, IDs y matriz
Modularidad	separación lib/semantic/ui
Usabilidad	editor, pestañas, filtros y exportación
Recuperación	estrategia ANTLR y control de cascadas
Mantenibilidad	Visitor, tipos discriminados y API de scopes

Cuadro 13.3: Atributos de calidad y evidencia concreta.

Capítulo 14

Trazabilidad con los requisitos

14.1 Reglas semánticas

Cuadro 14.1: Trazabilidad de reglas semánticas.

Área	Requisito	Evidencia
Tipos	aritmética numérica y concatenación	<code>typeSystem.ts</code> , SEM004
Tipos	lógica booleana	<code>isLogicalOperand</code> , pruebas
Tipos	comparación compatible	<code>isComparable</code>
Tipos	asignación e inicialización	<code>isAssignable</code> , SEM003
Tipos	arreglos homogéneos	<code>commonType</code> , SEM017
Scopes	nombre declarado y visible	<code>resolve</code> , SEM001
Scopes	redeclaración local	<code>declare</code> , SEM002
Scopes	uso antes de inicialización	estado <code>initialized</code> , SEM023
Funciones	aridad y argumentos	SEM006, SEM007
Funciones	retorno	inferencia y SEM008
Funciones	recursión y closures	hoisting y <code>captured</code>
Control	condiciones booleanas	SEM005
Control	<code>break</code> , <code>continue</code> y <code>return</code>	pilas y SEM009–SEM011
Control	alcanzabilidad	<code>flowAnalysis.ts</code> , SEM018
Clases	miembros y <code>this</code>	<code>lookup</code> , SEM012, SEM013
Clases	constructor	firmas y SEM014
Clases	herencia	<code>classId</code> , SEM020
Arreglos	índice y receptor	SEM015, SEM016
Switch	discriminante comparable	SEM021

14.2 Requisitos de construcción

Requisito	Evidencia	Estado
Parser generado	gramática y <code>src/generated</code>	Cubierto
Recorrido Visitor	<code>semanticVisitor.ts</code>	Cubierto
Árbol visual	CST y árbol semántico	Cubierto
Casos de prueba	siete suites y ejemplos	Cubierto
Tabla de símbolos	<code>scopes.ts</code> y panel UI	Cubierto
IDE	React/Vite y Electron	Cubierto
Documentación	informe, matriz y decisiones	Cubierto
Contribuciones	historial real de Git	Revisión manual

Cuadro 14.2: Cobertura de requisitos de construcción.

14.3 Correspondencia ponderada

Componente	Puntos	Evidencia
IDE	15	editor, fases, resultados y exportaciones
Parser y semántica	60	ANTLR, Visitor, tipos, diagnósticos, árboles y pruebas
Tabla de símbolos	25	inserción, recuperación, actualización, scopes y closures

Cuadro 14.3: Correspondencia resumida con la rúbrica del Proyecto 1.

14.4 Contribuciones

La autoría individual no puede inferirse con fiabilidad a partir del estado final de un archivo. Debe comprobarse mediante commits reales. La documentación no altera ni inventa atribuciones que el historial no respalde.

Capítulo 15

Auditoría técnica y decisiones

15.1 Hallazgos corregidos

Cuadro 15.1: Hallazgos y correcciones arquitectónicas.

Área	Hallazgo	Corrección
Clases	mapa global filtraba clases locales	identidad <code>classId</code> y resolución léxica
Instancias	igualdad basada solo en nombre	tipo conserva identidad de declaración
Campos	inferencia dependía del orden	campos antes que métodos
Funciones	retorno ausente tratado como void	tipos observados e inferencia común
Referencias	faltaban usos de clase y miembros	propagación de <code>symbolId</code>
Símbolos	no había actualización pública	<code>updateSymbol</code> con invariantes
Gramática	controles aceptaban cuerpos sin bloque	reglas alineadas y ANTLR regenerado
Ejemplos	fragmentos incompatibles con llaves	fixtures y pruebas actualizados
UI	resultados extensos competían con editor	IDE neobrutalista con dock y paneles
Documentación	rutas y cifras antiguas	matriz, informe y materiales actualizados

15.2 Decisión sobre float

Se conserva `float` como extensión mínima exigida por el enunciado. La promoción de entero a decimal es segura; la conversión inversa requiere una operación explícita que el lenguaje actual no ofrece.

15.3 Decisión sobre switch

Se priorizan gramática y ejemplos: discriminante escalar con casos comparables. Exigir booleano invalidaría el ejemplo entero oficial. La discrepancia queda documentada y aislada en `DECISIONES\_SEMANTICAS.md`.

15.4 Identidad de clases

Separar nombre e identidad es la decisión más importante de la revisión. El nombre responde a visibilidad; el ID responde a igualdad nominal. Confluir ambos conceptos había causado fugas de scope y compatibilidad falsa.

15.5 Inferencia independiente del orden

Un método puede referir un campo declarado más adelante porque los miembros se preparan antes del recorrido de cuerpos. El resultado presentado vuelve al orden fuente para no confundir al usuario. Se separa así orden de análisis de orden visual.

15.6 Omisión de semántica tras errores previos

La recuperación del parser es útil para informar sintaxis, pero sus nodos artificiales no son una base confiable para tipos. Omitir semántica reduce falsos positivos y explica explícitamente el motivo mediante `skipReason`.

Capítulo 16

Instalación, uso y mantenimiento

16.1 Requisitos

- Node.js 20.19 o posterior, o 22.12 o posterior;
- npm 9 o posterior;
- Java/JRE 11 o posterior para regenerar ANTLR;
- navegador moderno;
- Windows x64, macOS (Intel/Apple Silicon) o Linux x64 para los ejecutables configurados;
- distribución LaTeX con TikZ para recompilar este informe.

16.2 Instalación

```
cd compiscript-ide-work  
npm install  
npm run generate  
npm run dev
```

Después se abre <http://localhost:3000>. En ejecuciones ordinarias no es necesario regenerar si la gramática no cambió.

16.3 Uso del IDE

1. seleccionar la fase;
2. elegir un ejemplo, escribir código o cargar `.cps`;
3. ejecutar el análisis;
4. revisar aceptación y conteos;

5. inspeccionar diagnósticos, símbolos, scopes y árboles;
6. exportar evidencia si se necesita.

16.4 Uso de la CLI

```
npm run cli -- examples/semantic/valid_complete.cps --mode semantic
npm run cli -- examples/semantic/errors_types.cps --mode semantic
npm run cli:semantic-valid
npm run cli:semantic-errors
npm run cli:semantic-symbols
```

16.5 Construcción

```
npm run build
npm run desktop
npm run exe:portable    # Windows portable
npm run exe:installer   # Windows NSIS
npm run exe:mac         # macOS zip + dmg (x64/arm64)
npm run exe:linux       # Linux AppImage (x64)
```

16.6 Modificación de la gramática

Toda modificación requiere:

1. editar `src/grammars/Compiscript.g4`;
2. ejecutar `npm run generate`;
3. corregir errores TypeScript causados por cambios de contextos;
4. actualizar reglas semánticas afectadas;
5. agregar pruebas y ejemplos;
6. actualizar matriz y documentación.

16.7 Modificación de una regla semántica

El cambio debe actualizar como mínimo implementación, caso válido, caso inválido y trazabilidad. Si se crea un diagnóstico, se añade al tipo unión y catálogo. Si cambia la estructura de símbolos, se revisan serialización y paneles de UI.

16.8 Compilación del informe

Desde docs/informe:

```
latexmk -pdf -interaction=nonstopmode INFORME_PROYECTO_01.tex
```

Se requieren dos pasadas para resolver tabla de contenido, figuras, tablas y referencias. `latexmk` las ejecuta automáticamente.

Capítulo 17

Resultados, limitaciones y trabajo futuro

17.1 Resultados obtenidos

El proyecto entrega un frente reutilizable y observable. La gramática genera reconocedores reales; el orquestador controla transiciones; la semántica produce tipos, símbolos y diagnósticos; y la UI presenta el resultado sin duplicar reglas.

La tabla de símbolos demuestra las operaciones requeridas y agrega referencias, capturas e identidad. El árbol semántico permite observar el significado calculado. Los ejemplos y 115 pruebas convierten los requisitos en evidencia repetible.

17.2 Limitaciones

- no existe análisis entre múltiples archivos;
- no se modela sobrecarga de funciones;
- no hay genéricos, interfaces ni tipos unión;
- el análisis de flujo no construye un CFG completo;
- `try/catch` modela alcance, no excepciones dinámicas;
- el bundle web principal conserva una advertencia de tamaño;
- ninguno de los tres empaquetados (Windows, macOS, Linux) usa firma comercial;
- la política de `switch` depende de una decisión documentada;
- no existe navegación interactiva a declaración todavía.

17.3 Rendimiento

El análisis es predominantemente lineal respecto al tamaño del CST, salvo búsquedas de herencia y resolución por profundidad de scope. La resolución cuesta $O(h)$, donde h es la altura del árbol de ámbitos. En programas académicos esa altura es pequeña. Los mapas por scope hacen constante la búsqueda local promedio.

17.4 Seguridad y privacidad

La aplicación web procesa el código localmente. Las descargas usan blobs y URLs temporales. Electron mantiene Node desactivado en el renderer, aislamiento de contexto y sandbox. No se evalúa el código Compiscript ni se ejecutan comandos provenientes de la entrada.

17.5 Trabajo futuro

1. construir un grafo de flujo de control completo;
2. añadir navegación de uso a declaración;
3. crear una representación intermedia tipada;
4. incorporar interpretación segura;
5. realizar análisis interprocedural e interarchivo;
6. mejorar sugerencias con fragmentos y subrayado;
7. dividir vistas mediante carga dinámica;
8. ampliar empaquetado multiplataforma;
9. medir cobertura estructural y mutation testing.

Capítulo 18

Conclusiones

1. La gramática generada proporciona una base reproducible para el frente del compilador y evita implementar manualmente lexer y parser.
2. Separar análisis léxico, sintáctico y semántico reduce cascadas y permite explicar con precisión en qué fase falla una entrada.
3. La tabla de símbolos es el eje que conecta nombre, tipo, scope, declaración, referencia, inicialización y captura.
4. La identidad nominal de clases debe representar declaraciones y no únicamente textos; esta corrección elimina incompatibilidades sutiles.
5. La inferencia de campos y retornos requiere separar recolección de declaraciones y recorrido de cuerpos.
6. Los tipos `unknown` y `error` son herramientas de control de propagación, no tipos visibles ordinarios del lenguaje.
7. El análisis de flujo aporta información que no puede inferirse de una producción aislada, como retorno garantizado y alcanzabilidad.
8. Un contrato de resultado común evita discrepancias entre UI, CLI, Electron y pruebas.
9. La visualización conjunta de CST, árbol semántico, scopes y símbolos convierte el IDE en una herramienta pedagógica.
10. Las 115 pruebas y los archivos de demostración permiten repetir la evidencia técnica y proteger regresiones.

El resultado es una plataforma coherente para estudiar cómo un compilador pasa de texto a significado. La arquitectura está preparada para que una etapa posterior consuma tipos, símbolos y árboles sin reimplementar las fases ya verificadas.

Apéndice A

Programa integral de demostración

```
class Caja {
  let valor: integer;

  function constructor(valor: integer) {
    this.valor = valor;
  }

  function sumar(delta: integer): integer {
    return this.valor + delta;
  }
}

function ejecutar(): integer {
  let caja: Caja = new Caja(40);
  let ajustes: integer[] = [1, 2, 3];
  let base: integer = caja.valor;

  function aplicar(indice: integer): integer {
    return base + ajustes[indice];
  }

  return aplicar(1);
}

print(ejecutar());
```

Este programa genera scopes global, clase, métodos, función y closure. La tabla incluye clase, campos, métodos, variables, parámetros y funciones. **base** y **ajustes** quedan capturados por **aplicar**.

Apéndice B

Ejemplos de diagnósticos

B.1 Tipos

```
let activo: boolean = true;
let total: integer = activo;           // SEM003
let resultado = activo * 2;           // SEM004
if (total) { print(total); }          // SEM005
let mezcla = [1, "dos"];              // SEM017
```

B.2 Scopes

```
let pendiente: integer;
print(pendiente);                     // SEM023
print(noExiste);                     // SEM001
let repetida: integer = 1;
let repetida: integer = 2;           // SEM002
```

B.3 Funciones

```
function convertir(x: integer): string {
    return x;                         // SEM008
}
convertir();                          // SEM006
convertir("uno");                     // SEM007
let valor: integer = 1;
valor();                              // SEM014
return 1;                             // SEM009
```

B.4 Clases

```
class A {
```

```

    let valor: integer;
}
let a: A = new A();
print(a.inexistente);           // SEM012
print(this);                    // SEM013
let b = new NoExiste();         // SEM014

```

B.5 Flujo y arreglos

```

break;                          // SEM010
continue;                       // SEM011
let datos: integer[] = [1, 2, 3];
print(datos[1.5]);              // SEM015
let escalar: integer = 1;
print(escalar[0]);              // SEM016

```


Apéndice C

Inventario de módulos

Cuadro C.1: Inventario técnico del núcleo.

Archivo	Responsabilidad
<code>src/grammars/Compiscript.g4</code>	fuelle sintáctica y léxica
<code>src/generated/CompiscriptLexer.ts</code>	lexer generado
<code>src/generated/CompiscriptParser.ts</code>	parser generado
<code>src/lib/analyze.ts</code>	orquestador y normalización
<code>src/lib/antlrErrors.ts</code>	listeners y mensajes
<code>src/lib/treeFormat.ts</code>	conversión visual del CST
<code>src/lib/types.ts</code>	contratos comunes
<code>src/semantic/semanticTypes.ts</code>	representación de tipos
<code>src/semantic/typeSystem.ts</code>	asignabilidad y operadores
<code>src/semantic/symbols.ts</code>	modelo de símbolo
<code>src/semantic/scopes.ts</code>	árbol de scopes y resolución
<code>src/semantic/declarationVisitor.ts</code>	clases, miembros y firmas
<code>src/semantic/semanticVisitor.ts</code>	reglas y árbol semántico
<code>src/semantic/flowAnalysis.ts</code>	retornos y alcanzabilidad
<code>src/semantic/diagnostics.ts</code>	catálogo y creación
<code>src/semantic/ast.ts</code>	nodos semánticos
<code>src/cli/run.ts</code>	interfaz de consola
<code>src/ui/App.tsx</code>	coordinación del frontend
<code>src/ui/components/EditorTabs.tsx</code>	editor Monaco y archivos
<code>src/ui/components/RightDock.tsx</code>	navegación de resultados
<code>src/ui/components/SymbolTablePanel.tsx</code>	tabla de símbolos

Archivo	Responsabilidad
<code>src/ui/components/ScopeTreePanel.tsx</code>	árbol de scopes
<code>src/ui/components/SemanticTreePanel.tsx</code>	árbol semántico
<code>electron/main.cjs</code>	ventana de escritorio

Apéndice D

Inventario de comandos

Comando	Propósito
<code>npm install</code>	instalar dependencias fijadas
<code>npm run generate</code>	regenerar ANTLR
<code>npm run dev</code>	iniciar Vite
<code>npm run check</code>	comprobar TypeScript
<code>npm test</code>	ejecutar 115 pruebas
<code>npm run test:examples</code>	ejecutar casos semánticos en disco
<code>npm run build</code>	construir producción web
<code>npm run preview</code>	servir el build
<code>npm run cli - archivo -mode semantic</code>	analizar desde terminal
<code>npm run desktop</code>	construir y abrir Electron
<code>npm run exe:portable</code>	crear portable Windows x64
<code>npm run exe:installer</code>	crear instalador NSIS
<code>npm run exe:mac</code>	crear zip/dmg macOS (x64/arm64)
<code>npm run exe:linux</code>	crear AppImage Linux x64

Apéndice E

Glosario

ANTLR. Generador de analizadores a partir de gramáticas.

Ámbito o scope. Región que determina visibilidad de declaraciones.

AST. Árbol abstracto que elimina detalles superficiales.

Atributo. Información sintetizada o heredada durante el análisis.

Closure. Función junto con variables capturadas de scopes externos.

CST. Árbol concreto derivado directamente de la gramática.

Diagnóstico. Error o advertencia localizado y categorizado.

Hoisting. Registro anticipado de declaraciones dentro de un scope.

Inferencia. Deducción de tipo a partir de expresiones.

Lexema. Texto concreto reconocido.

Parser. Componente que valida una secuencia de tokens.

Shadowing. Declaración hija que oculta temporalmente una homónima.

Símbolo. Entidad declarada con identidad, tipo y ubicación.

Token. Categoría léxica asignada a un lexema.

Visitor. Patrón para ejecutar operaciones sobre nodos de un árbol.

Apéndice F

Ficha técnica

Elemento	Valor
Lenguaje analizado	Compiscript
Entrada	texto y archivos .cps
Generador	ANTLR 4 mediante antlr4ts
Lenguaje de implementación	TypeScript
Frontend	React 19 y Vite 6
Escritorio	Electron 43
Regla inicial	program
Modos	lexer, parser y semantic
Tipos	primitivos, arreglos, funciones, clases e instancias
Scopes	global, bloque, función, clase, ciclo, catch y switch
Diagnósticos	SEM001 – SEM023
Pruebas	115 en 7 suites
Salidas	UI, CLI, CSV, JSON, TXT y árboles
Plataforma empaquetada	Windows x64, macOS (x64/arm64) y Linux x64
Alcance	frente hasta análisis semántico

Cuadro F.1: Ficha resumida del producto.

DragonsSlayers 2.0

Proyecto 1 – Construcción de Compiladores CC-3032

Informe ampliado a partir de la implementación y documentación verificables.